

A Project Report
On
Predictive Analytics and Explainable AI for Hospital Readmission Risk



**University
of Regina**

Faculty of Graduate Studies and Research

CS 719 - Data Science Project

Winter 2026

Prepared By:

Raj Panchal [200490453]

Instructor:

Howard J. Hamilton

Table of Contents

	Page No.
1. Introduction	1
1.1 Main Idea of the Project	
1.2 Type of Research Problem	
1.3 Problem Specification	
1.4 Why It Is Useful	
2. Background	2
2.1 Existing Techniques Relevant to the Project	
2.2 Analysis of Strengths and Weaknesses	
2.3 Learning Resources	
3. Approach	4
3.1 Data Source	
3.2 Methodology	
3.3 Worked Example	
3.4 Software Description	
3.5 Processing Time and Space	
4. Implementation	9
4.1 Software Stack and Versions	
4.2 Module Structure	
4.3 Three Crucial Code Snippets	
5. Results of Running Software	11
5.1 Dataset and Experiments	
5.2 Model Training and Evaluation	
5.3 Threshold Optimization	
5.4 Explainability Results	
5.5 LACE Index vs. XGBoost Comparison	
5.6 Streamlit Dashboard	
5.7 Connection to Recent Research	
6. Conclusions	18
7. Suggestions for Future Research	19
8. References	20

1. Introduction

1.1 Main Idea of the Project

Hospital readmissions within 30 days of discharge are a major concern for healthcare systems around the world. They usually point to gaps in the quality of care and drive up costs significantly. In the United States, preventable readmissions are estimated to cost billions of dollars each year, which led to the creation of the Hospital Readmissions Reduction Program (HRRP) under the Affordable Care Act. Through the HRRP, hospitals with high 30-day readmission rates face financial penalties, which has pushed the industry to look for better ways to identify at-risk patients before discharge.

This project focused on building a predictive model to estimate whether a diabetic patient would be readmitted within 30 days. I used the Diabetes 130-US Hospitals dataset, which contains 101,766 patient encounters from 130 U.S. hospitals between 1999 and 2008. I trained and compared three classifiers, Logistic Regression, Random Forest, and XGBoost, and paired the best model with explainability methods like Permutation Feature Importance (PFI) and Partial Dependence Plots (PDP) so that the results would be interpretable for clinical decision-makers who may not have a machine learning background.

I also ran a head-to-head comparison between the trained XGBoost model and the LACE Index, which is a rule-based scoring tool commonly used in Canadian hospitals for readmission risk screening [van Walraven et al., 2010]. Both approaches were evaluated on the exact same held-out test set with the same ground truth labels, so the comparison was direct and fair.

1.2 Type of Research Problem

This project addresses a supervised binary classification problem in applied data science. Given historical patient encounter data, the task was to predict whether a patient would be readmitted to the hospital within 30 days of discharge. The target variable was derived from the original three-class label (readmitted in less than 30 days, more than 30 days, or not readmitted) by converting it into a binary outcome: 1 for readmitted within 30 days and 0 for everything else.

Rather than proposing a new algorithm, the project focused on following standard data science practices, including data preprocessing, model development and evaluation, threshold optimization for clinical utility, and explainability of predictions.

1.3 Problem Specification

- Input:
 - The Diabetes 130-US Hospitals dataset containing structured tabular data with 50 features
 - Patient demographics, admission details, ICD-9 diagnosis codes, medications, and hospital utilization variables
- Output:
 - A binary prediction (0 = not readmitted within 30 days, 1 = readmitted within 30 days)
 - Model evaluation metrics: Accuracy, Precision, Recall, F1-score, and ROC-AUC
 - Feature importance rankings using Permutation Feature Importance (PFI)

- Partial Dependence Plots showing how key features influence readmission risk
- LACE Index scores for every test patient, compared directly against the ML model
- An interactive Streamlit dashboard for real-time patient risk assessment

1.4 Why It Is Useful

Predicting hospital readmissions has direct clinical value. If a hospital can figure out which patients are likely to come back within 30 days, it can focus its resources on those patients through follow-up calls, medication reviews, or scheduling outpatient visits. Missing a high-risk patient is far more costly than a false alarm: a flagged patient simply gets extra attention, while a missed one may end up back in the emergency room.

Explainability is just as important. Clinicians are unlikely to trust a model that just outputs a risk score without explaining why. By showing which features drove each prediction and how they compare to the population average, the system helps clinicians understand why a patient was flagged instead of blindly trusting a black-box model.

2. Background

2.1 Existing Techniques Relevant to the Project

Several machine learning and explainability techniques were relevant to this project. I reviewed each of these before deciding which to use, drawing on course material from CS 719, the scikit-learn documentation [Pedregosa et al., 2011], and the textbook "Interpretable Machine Learning" by Christoph Molnar [Molnar, 2022].

Logistic Regression

Logistic Regression is a linear classifier that models the probability of a binary outcome as a logistic function of the input features. It is inherently interpretable because its coefficients directly indicate the direction and magnitude of each feature's influence. However, it assumes a linear relationship between features and the log-odds of the outcome, which limits its ability to capture complex patterns in the data. I used it as a baseline model to compare against more complex classifiers.

Random Forest

Random Forest is an ensemble method that trains multiple decision trees on random subsets of the data and features, then averages their predictions [Breiman, 2001]. It handles non-linearity and feature interactions well and is relatively resistant to overfitting. The downside is that the ensemble of hundreds of trees is difficult to interpret directly, which is why post-hoc explainability techniques like PFI are needed.

XGBoost (Extreme Gradient Boosting)

XGBoost is a gradient boosting framework that builds trees sequentially, where each new tree corrects the errors of the previous ones [Chen and Guestrin, 2016]. It is widely used in data science competitions and applied research because of its strong performance on structured tabular data. Like Random Forest, it is a black-box model, but its predictions can be explained using techniques like PFI and PDP. I selected XGBoost as one of the three candidate models because of its known effectiveness on healthcare datasets.

SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE addresses class imbalance by generating synthetic samples of the minority class rather than simply duplicating existing ones [Chawla et al., 2002]. It works by interpolating between existing minority-class samples and their nearest neighbors. I applied SMOTE only to the training set (not the test set) to avoid data leakage, which is a common mistake in readmission prediction studies as noted by Guo et al. [2025].

Permutation Feature Importance (PFI)

PFI measures the importance of each feature by randomly shuffling its values and observing how much the model's performance drops [Molnar, 2022]. A large drop means the feature is important. PFI is model-agnostic, meaning it works with any classifier, and it measures importance on the actual test set rather than relying on training-time statistics. I used PFI scored on ROC-AUC to identify the top predictors of readmission.

Partial Dependence Plots (PDP)

PDPs show how the predicted outcome changes as a single feature varies while all other features are held constant [Molnar, 2022]. They provide an intuitive visualization of the relationship between a feature and the prediction. For example, a PDP for "number of prior inpatient visits" can show whether readmission risk increases linearly or levels off after a certain point. I generated PDPs for the top features identified by PFI.

LACE Index

The LACE Index is a rule-based clinical scoring system developed by van Walraven et al. [2010] to predict the risk of death or unplanned readmission within 30 days of discharge. It uses four factors: Length of stay (L), Acuity of admission (A), Comorbidity as measured by the Charlson Comorbidity Index (C), and Emergency department visits in the past six months (E). A LACE score of 10 or higher indicates high risk. It is widely used in Canadian hospitals because of its simplicity: it requires no model training and can be computed by hand. I included it in this project as a clinical baseline to benchmark against the ML model.

2.2 Analysis of Strengths and Weaknesses

- Strengths of the approach used in this project:
 - Uses a real hospital dataset with actual patient records, not synthetic data.
 - Applies PFI and PDP, which are model-agnostic and work well for clinical tabular data.
 - Uses a small, focused set of models (three classifiers) rather than testing dozens, keeping the analysis clear and reproducible.
 - Produces explanations for one final selected model, avoiding unnecessary complexity.
 - Compares the ML model against a real clinical baseline (LACE Index) on the same test set.
 - Includes an interactive dashboard that makes the results accessible to users without a machine learning background.
- Weaknesses and limitations:
 - Uses only one dataset, which may limit how well the results generalize to other hospitals or patient populations.
 - The data spans 1999 to 2008, so clinical practices and coding standards may have changed since then.
 - Explainability results from PFI and PDP are interpretive, not causal. A strong association between a feature and readmission does not prove that the feature causes readmission.

- The Charlson Comorbidity Index for the LACE comparison had to be approximated from diagnosis category columns, since the dataset does not include a direct CCI field.

2.3 Learning Resources

The techniques used in this project were implemented using knowledge gained from several sources:

- Course material from CS 719, particularly the guidelines for each project deliverable and the overall data science workflow.
- Scikit-learn documentation [Pedregosa et al., 2011] for model training, evaluation metrics, and preprocessing utilities.
- "Interpretable Machine Learning" by Christoph Molnar [Molnar, 2022], an open-access textbook that covers PFI, PDP, and other explainability methods in detail.
- The XGBoost documentation [Chen and Guestrin, 2016] for understanding gradient boosting parameters and tuning strategies.
- Peer-reviewed research articles listed in Section 8, particularly Guo et al. [2025] for their systematic comparison of sampling strategies and explainability techniques on the same dataset.

3. Approach

3.1 Data Source

The project used the Diabetes 130-US Hospitals Dataset, publicly available from the UCI Machine Learning Repository [Strack et al., 2014]. The dataset contains 101,766 patient encounters from 130 U.S. hospitals collected between 1999 and 2008. Each record represents a single hospital visit for a diabetic patient and includes 50 features covering demographics (age, race, gender), admission details (type, source, discharge disposition), diagnoses (three ICD-9 codes), medications (23 diabetes-related drugs), lab results, and hospital utilization history (prior inpatient, outpatient, and emergency visits).

The original target variable had three categories: readmitted in less than 30 days, readmitted in more than 30 days, and not readmitted. I converted this into a binary target: 1 for readmitted within 30 days (11,314 encounters, 11.39%) and 0 for everything else (88,026 encounters, 88.61%). This severe class imbalance was one of the main challenges of the project.

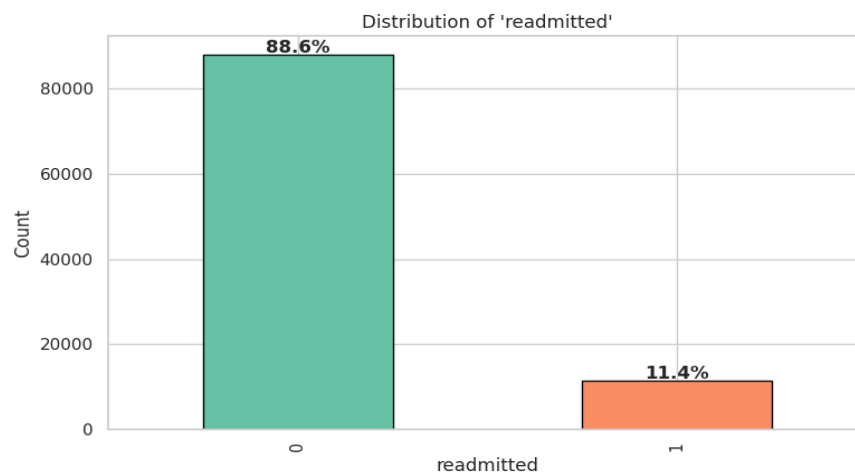


Fig 1. Binary target distribution showing the class imbalance (88.6% vs 11.4%).

3.2 Methodology

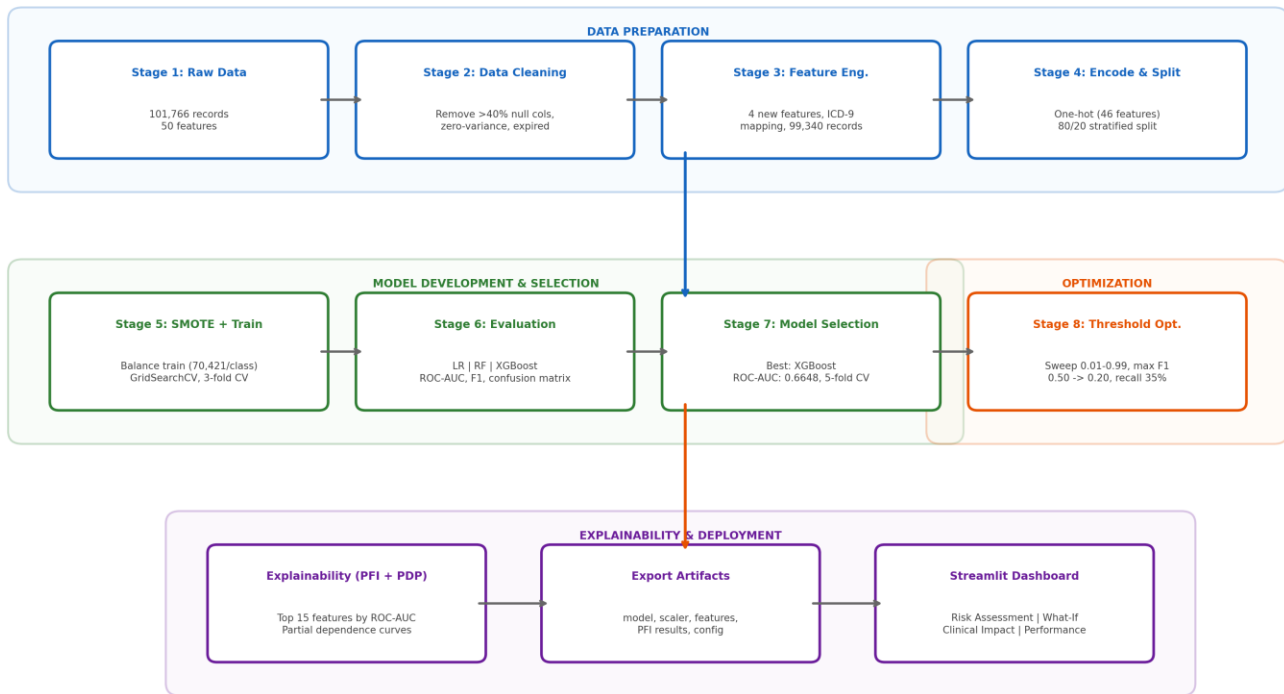


Fig 2. End-to-end pipeline showing all eight stages from raw data to deployment.

The project followed an end-to-end machine learning pipeline with eight stages:

- **Stage 1: Data Cleaning**

I removed columns with more than 40% missing values (weight, medical_specialty, max_glu_serum, A1Cresult), dropped two zero-variance columns (examide, citoglipton), and filtered out patients who were discharged to hospice or expired (since readmission is not applicable for them). After cleaning, 99,340 encounters remained with 42 columns.

- **Stage 2: Feature Engineering**

I created four new features from the existing data: total_visits (sum of outpatient, emergency, and inpatient visit counts), num_med_changed (count of medications that were adjusted during the visit), num_med_active (count of medications actively prescribed), and age_numeric (midpoint of the age bracket, e.g., [70-80) became 75). The three ICD-9 diagnosis code columns (diag_1, diag_2, diag_3) were mapped into nine clinical categories: Circulatory, Respiratory, Digestive, Diabetes, Injury, Musculoskeletal, Genitourinary, Neoplasms, and Other.

- **Stage 3: Encoding and Splitting**
Categorical features were one-hot encoded, producing a final feature matrix of 46 columns. The data was split into 80% training (79,472 encounters) and 20% testing (19,868 encounters) using stratified sampling to preserve the class distribution in both sets.
- **Stage 4: SMOTE on Training Set**
SMOTE [Chawla et al., 2002] was applied only to the training set to balance the classes, bringing both classes to 70,421 samples each. The test set was left untouched to ensure a realistic evaluation. All features were then scaled using StandardScaler.
- **Stage 5: Model Training and Hyperparameter Tuning**
Three classifiers were trained: Logistic Regression, Random Forest, and XGBoost. For each model, I ran GridSearchCV with 3-fold cross-validation scored on ROC-AUC to find the best hyperparameters. All three models used the same preprocessed data and the same train-test split.
- **Stage 6: Model Evaluation and Selection**
Models were evaluated on the held-out test set using ROC-AUC, accuracy, precision, recall, and F1-score. I also ran 5-fold stratified cross-validation on the training set to check stability. XGBoost was selected as the best model based on the highest test ROC-AUC (0.6648) and stable cross-validation scores.
- **Stage 7: Threshold Optimization**
The default classification threshold of 0.50 resulted in very low recall (only 4% of actual readmissions detected). I swept thresholds from 0.05 to 0.95 and found that a threshold of 0.20 maximized the F1-score, improving recall to 35% while keeping precision at 23%.
- **Stage 8: Explainability and Dashboard**
PFI and PDP were computed on the best model to identify and visualize the most important features. The trained model, scaler, and supporting data were exported as artifacts and loaded into an interactive Streamlit dashboard for real-time risk assessment.

3.3 Worked Example

To illustrate how the system works from input to output, here is a fully worked example for a medium-risk patient:

- **Sample Input:**
 - Age group: 60-70 (age_numeric = 65)
 - Prior inpatient visits: 2
 - Number of medications: 15
 - Time in hospital: 5 days
 - Number of diagnoses: 7
 - Discharge disposition: 1 (discharged to home)
 - Admission type: 1 (emergency)
 - Primary diagnosis category: Circulatory

Processing Steps:

First, the raw inputs are placed into a feature vector with 46 columns matching the training format. Numeric features (age_numeric=65, number_inpatient=2, num_medications=15, etc.) are placed directly. Categorical features are one-hot encoded: for example, diag_1="Circulatory" means the column diag_1_Circulatory is set to 1 and all other diag_1_* columns are set to 0. The feature vector is then scaled using the same StandardScaler that was fitted on the training data.

The scaled feature vector is passed to the trained XGBoost model, which outputs a predicted probability of readmission. For this patient, the predicted probability was 0.24 (24%). Since 0.24 exceeds the optimized threshold of 0.20, the binary prediction is 1 (likely to be readmitted), and the risk level is classified as MEDIUM RISK (between 0.20 and 0.30).

- Sample Output:
 - Predicted probability of readmission: 0.24 (24%)
 - Binary prediction: 1 (likely to be readmitted)
 - Risk level: MEDIUM RISK
 - Top contributing features: number_inpatient (2, above average), num_medications (15, above average), time_in_hospital (5, above average)

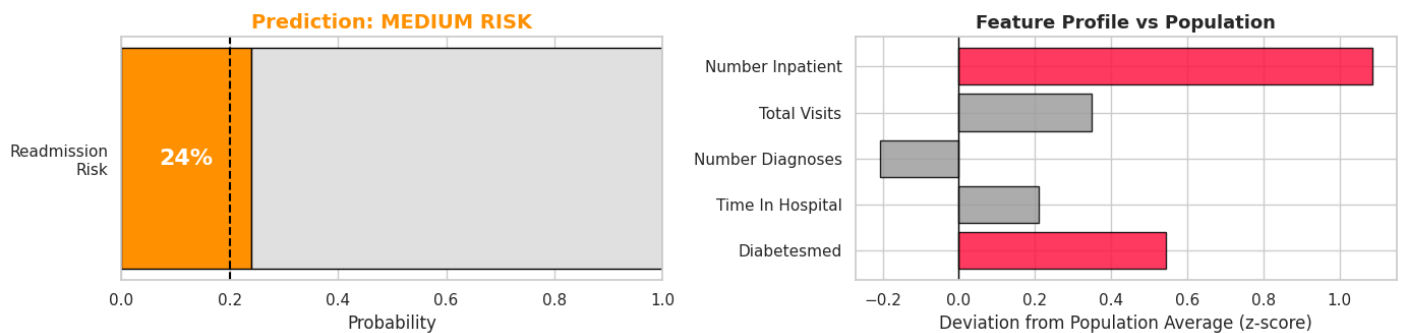


Fig 3. Notebook output for a medium-risk patient: risk gauge (left) and feature profile compared to the population average (right).

The LACE Index was also computed for the same patient using the dashboard's live comparison feature. As shown in Fig 4, LACE gives a score of 9 out of 19 (below the threshold of 10), classifying this patient as low risk. Meanwhile, the XGBoost model flags the same patient as medium risk at 24%. This shows a case where the ML model catches something that LACE misses.

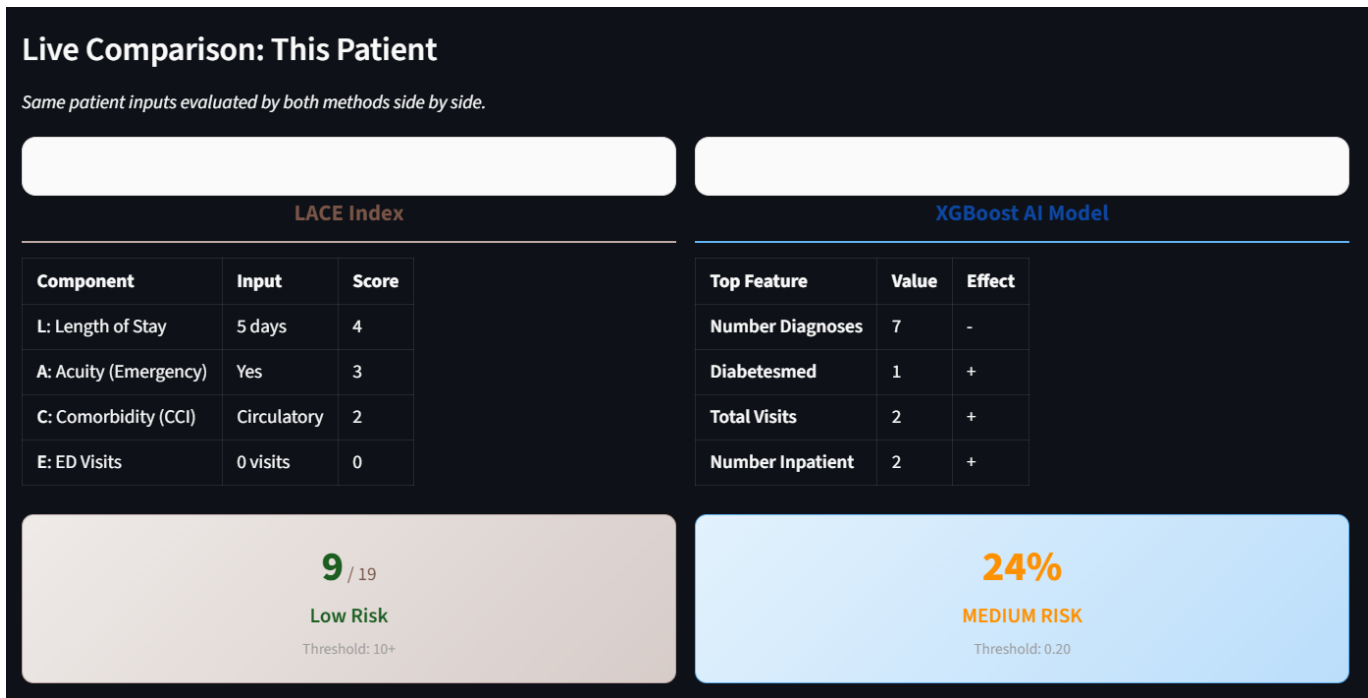


Fig 4. Live comparison from the Streamlit dashboard: LACE Index (score 9, low risk) vs. XGBoost AI model (24%, medium risk) for the same patient.

3.4 Software Description

The software produced in this project consists of two main components:

The first component is a Jupyter Notebook that contains the complete data science pipeline: data loading, cleaning, feature engineering, model training, evaluation, threshold optimization, explainability analysis, and the LACE comparison. The notebook is fully executable on Google Colab and produces all results, tables, and figures from scratch.

The second component is a Streamlit web dashboard that loads the trained model artifacts and has four tabs: (1) Risk Assessment, where you adjust patient parameters with sliders and get a risk score along with which features are driving the prediction and a suggested action plan; (2) Explainability and What-If, which shows global feature importance and lets you change patient inputs to see how the risk changes; (3) Clinical Impact, which covers the threshold optimization, a side-by-side LACE vs. XGBoost comparison for the current patient, and an estimated cost savings calculator; and (4) Model Performance, which shows the model comparison table and a summary of the dataset and methodology. The dashboard is deployed online and auto-updates from the GitHub repository on every push.

3.5 Processing Time and Space

The entire notebook, including data loading, cleaning, feature engineering, training three models with GridSearchCV, computing PFI (10 repeats), generating all plots, and running the LACE comparison, executed end-to-end in approximately 8 to 10 minutes on a Google Colab free-tier instance (2 vCPUs, 12.7 GB RAM). The most time-consuming steps were GridSearchCV for XGBoost (27 parameter combinations times 3 folds) and PFI computation (46 features times 10 repeats).

The dataset file (data.csv) is approximately 18.2 MB. The exported model artifacts total about 783 KB, of which the trained XGBoost model file is the largest at 776 KB. The Streamlit dashboard loads these artifacts at startup and responds to user interactions in under one second.

4. Implementation

4.1 Software Stack and Versions

The project was implemented entirely in Python. The notebook was developed and executed on Google Colab, and the dashboard was built with Streamlit and deployed via Streamlit Community Cloud. Table 1 lists the main packages used and their versions.

Table 1. Software packages and versions used in the project.

Package	Version	Purpose
Python	3.10	Programming language
pandas	2.0+	Data manipulation and analysis
NumPy	1.24+	Numerical computation
scikit-learn	1.3+	ML models, metrics, preprocessing
XGBoost	2.0+	Gradient boosting classifier
imbalanced-learn	0.11+	SMOTE for class balancing
Matplotlib	3.7+	Plotting and visualization
Seaborn	0.12+	Statistical visualizations
Streamlit	1.32+	Interactive web dashboard
joblib	1.3+	Model serialization
SciPy	1.11+	Statistical tests

4.2 Module Structure

The project consists of three main source files and a set of exported model artifacts. Table 2 summarizes the module structure.

Table 2. Module structure of the project.

File	Lines	Description
Notebook (.ipynb)	736 (code) + 24 markdown cells	End-to-end ML pipeline
streamlit_app.py	857	Interactive dashboard with 4 tabs
src/utls.py	630	Reusable helper functions (plotting, evaluation, preprocessing)
model_artifacts/	6 files (~783 KB)	Trained model, scaler, feature names, config, PFI results, stats

In total, the project contains approximately 2,223 lines of Python code across the notebook, dashboard, and utility module, plus 24 markdown cells of documentation in the notebook.

4.3 Three Crucial Code Snippets

Snippet 1: SMOTE and GridSearchCV Training Pipeline

This snippet shows how the training data was balanced with SMOTE and then used to tune the XGBoost classifier via grid search. The key design choice was applying SMOTE before GridSearchCV so that the cross-validation folds within GridSearchCV also see balanced data. The grid tested 27 combinations of learning rate, max depth, and number of estimators.

```
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

xgb_params = {
    'n_estimators': [100, 200, 300],
    'max_depth': [4, 6, 8],
    'learning_rate': [0.05, 0.1, 0.2]
}
xgb_search = GridSearchCV(
    XGBClassifier(random_state=42, eval_metric='logloss', n_jobs=-1),
    xgb_params, scoring='roc_auc', cv=3, n_jobs=-1
)
xgb_search.fit(X_train_sm, y_train_sm)
xgb = xgb_search.best_estimator_
```

Snippet 2: LACE Comorbidity Approximation from Diagnosis Categories

The LACE Index requires a Charlson Comorbidity Index (CCI) score, but the dataset does not include one directly. I approximated it using the one-hot encoded diagnosis category columns. Each diagnosis slot (diag_1, diag_2, diag_3) is checked for categories that correspond to CCI-weighted conditions: Diabetes carries a weight of 1, Neoplasms carries 2, and Respiratory and Genitourinary each carry 1. The total is capped at 5, matching the LACE scoring table.

```
def approx_cci_from_diag_cols(row):
    cci = 0
    for prefix in ['diag_1_', 'diag_2_', 'diag_3_']:
        if row.get(f'{prefix}Diabetes', 0) == 1: cci += 1
        if row.get(f'{prefix}Neoplasms', 0) == 1: cci += 2
        if row.get(f'{prefix}Respiratory', 0) == 1: cci += 1
        if row.get(f'{prefix}Genitourinary', 0) == 1: cci += 1
    return min(cci, 5)
```

Snippet 3: Threshold Optimization Sweep

At the default threshold of 0.50, the model barely flagged anyone as high risk (recall was only 4%). This snippet sweeps every threshold from 0.05 to 0.95, computes precision, recall, and F1 at each one, and picks the threshold that maximizes F1. The result was 0.20, which brought recall up to 35% while keeping precision at 23%. This single change made the model nine times better at catching actual readmissions.

```

y_prob = model.predict_proba(X_test)[: , 1]

thresholds = np.arange(0.05, 0.96, 0.01)
precisions, recalls, f1s = [], [], []

for t in thresholds:
    y_pred = (y_prob >= t).astype(int)
    precisions.append(precision_score(y_test, y_pred, zero_division=0))
    recalls.append(recall_score(y_test, y_pred, zero_division=0))
    f1s.append(f1_score(y_test, y_pred, zero_division=0))

best_idx = np.argmax(f1s)
best_threshold = thresholds[best_idx]

```

5. Results of Running Software

5.1 Dataset and Experiments

All experiments were run on the Diabetes 130-US Hospitals dataset. After data cleaning, 99,340 patient encounters remained. After one-hot encoding, the feature matrix had 46 columns. The data was split into a training set of 79,472 encounters and a test set of 19,868 encounters using an 80/20 stratified split. SMOTE was applied to the training set only, bringing both classes to 70,421 samples each. All evaluation metrics reported below are computed on the untouched test set.

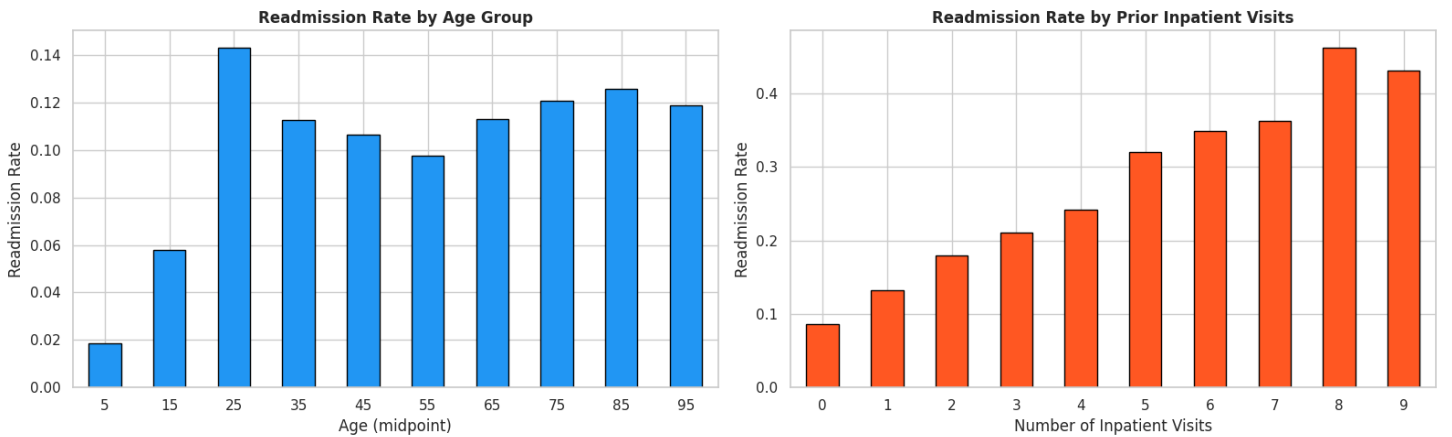


Fig 5. Readmission rate by age group (left) and by number of prior inpatient visits (right). Prior inpatient visits show a much stronger relationship with readmission than age.

5.2 Model Training and Evaluation

Table 3 summarizes the GridSearchCV results for all three models. XGBoost achieved the highest test ROC-AUC (0.6648), and all three models showed stable cross-validation scores with low standard deviation.

Table 3. Model comparison on the held-out test set.

Model	Best Parameters	Test ROC-AUC	Test Accuracy	Test F1
Logistic Regression	C=0.01, solver=lbfgs	0.6418	64.1%	0.7053

Random Forest	n_est=200, depth=20	0.6470	88.5%	0.8374
XGBoost	lr=0.05, depth=8, n_est=100	0.6648	88.6%	0.8406

I selected XGBoost as the best model based on its highest ROC-AUC score. While Random Forest had comparable accuracy and F1, XGBoost had a higher ROC-AUC, which matters more than accuracy when classes are imbalanced.

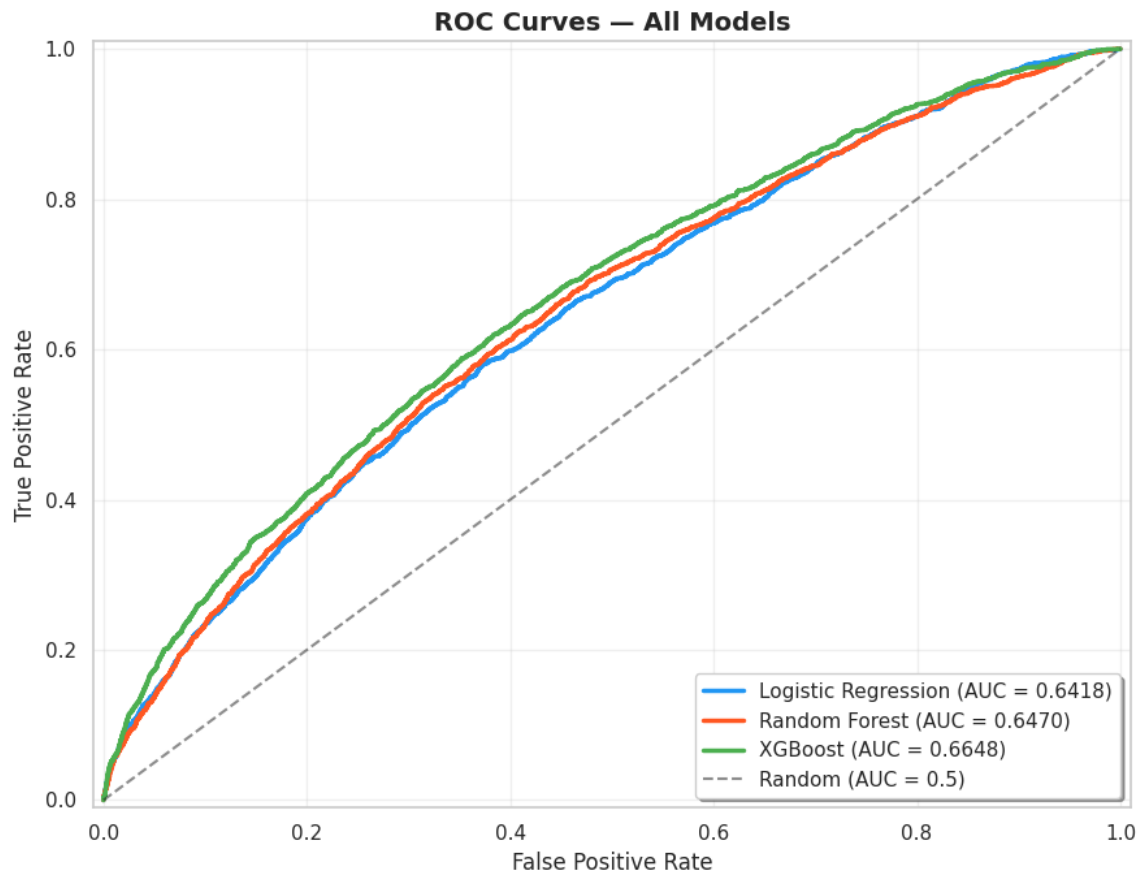


Fig 6. ROC curves for all three models on the test set. XGBoost achieves the highest AUC (0.6648).

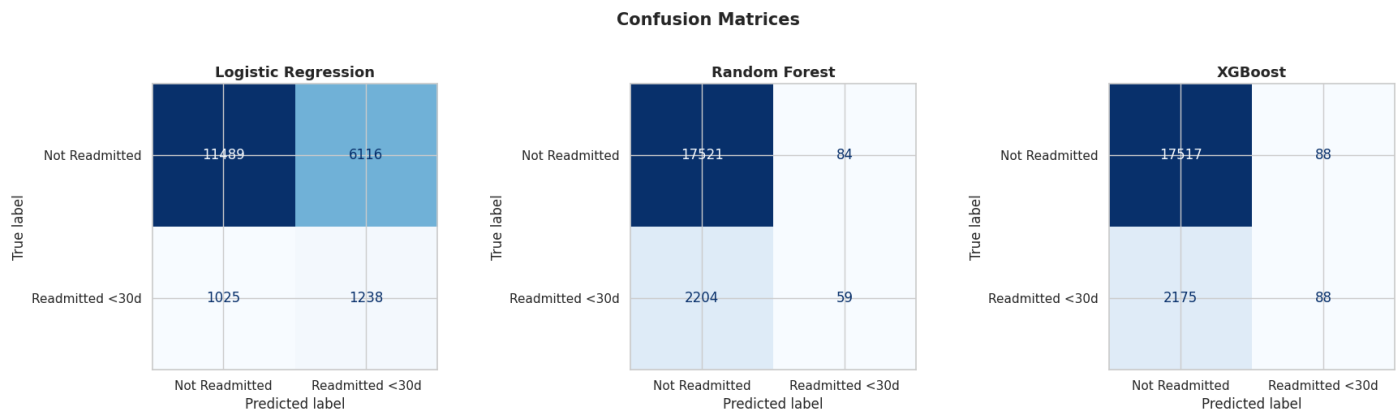


Fig 7. Confusion matrices for all three models at the default threshold (0.50).

The confusion matrices at the default 0.50 threshold (Fig 7) show a typical issue with imbalanced data: both Random Forest and XGBoost achieve high accuracy by mostly predicting the majority class. Logistic Regression has lower accuracy but actually detects more readmissions. This motivated the threshold optimization in the next section.

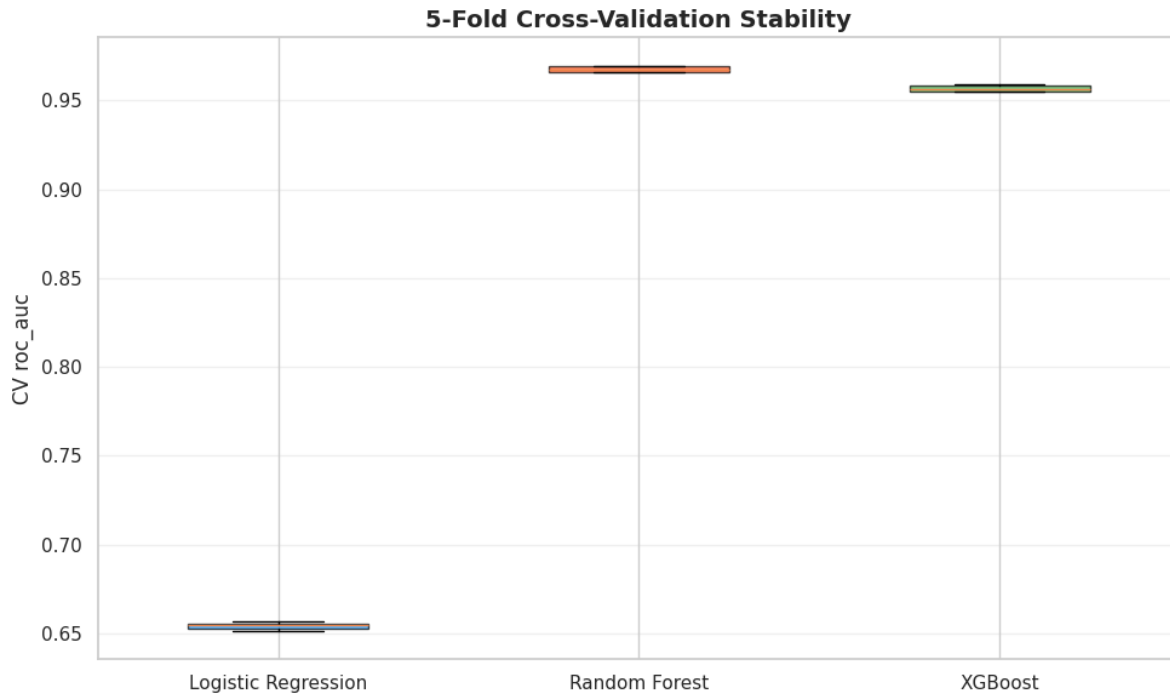


Fig 8. Five-fold stratified cross-validation stability (ROC-AUC) for all three models.

All three models stayed within a narrow range across folds, which confirms the results are not dependent on a particular train-test split.

5.3 Threshold Optimization

At the default threshold of 0.50, XGBoost caught only 4% of actual readmissions (recall = 0.04). This is unacceptable in a clinical setting where missing a high-risk patient has real consequences. I swept thresholds from 0.05 to 0.95 and selected 0.20 as the optimal threshold because it maximized the F1-score on the test set. Table 4 compares the two thresholds.

Table 4. Effect of threshold optimization on XGBoost performance.

Metric	Threshold = 0.50	Threshold = 0.20	Change
Precision	50%	23%	-27%
Recall	4%	35%	+31%
F1-Score	0.07	0.28	+0.21
Accuracy	89%	79%	-10%

The trade-off here is: lowering the threshold increases false positives (more patients flagged who will not actually be readmitted), but greatly reduces false negatives (fewer missed high-risk patients). In healthcare, this

trade-off is worth making because a flagged patient simply gets extra follow-up care, while a missed patient may end up back in the emergency room.

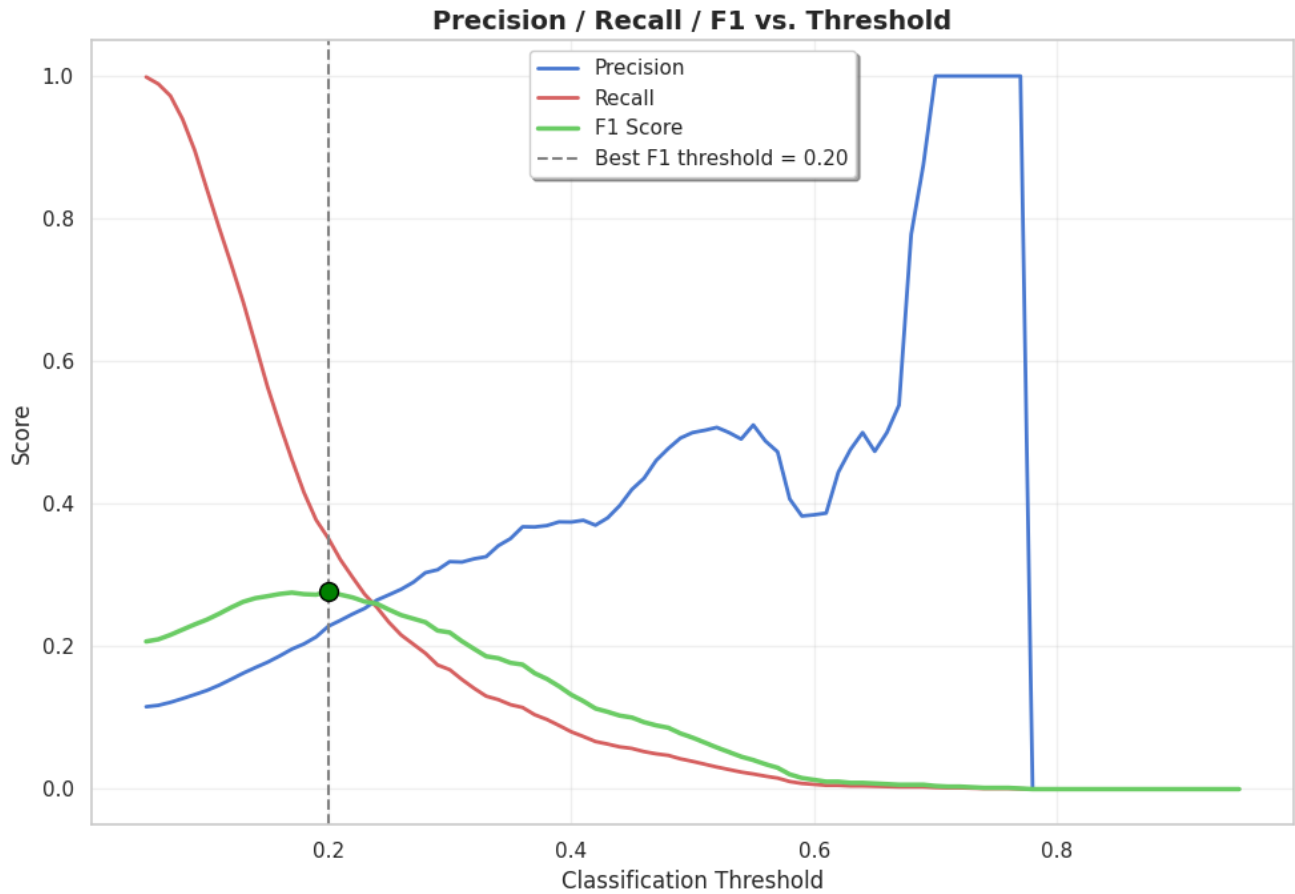


Fig 9. Threshold sweep showing precision, recall, and F1-score across thresholds. The vertical dashed line marks the optimal threshold at 0.20.

5.4 Explainability Results

Permutation Feature Importance was computed on the test set with 10 repeats per feature, scored on ROC-AUC. The top five features are shown in Table 5.

Table 5. Top 5 features by Permutation Feature Importance (ROC-AUC drop).

Rank	Feature	ROC-AUC Drop	Why It Matters
1	number_inpatient	5.71%	Prior hospital use is the strongest signal
2	discharge_disposition_id	4.50%	Where the patient goes after discharge
3	total_visits	0.79%	Combined outpatient + ER + inpatient visits
4	number_diagnoses	0.38%	More diagnoses = higher complexity
5	admission_type_id	0.32%	Emergency vs. elective

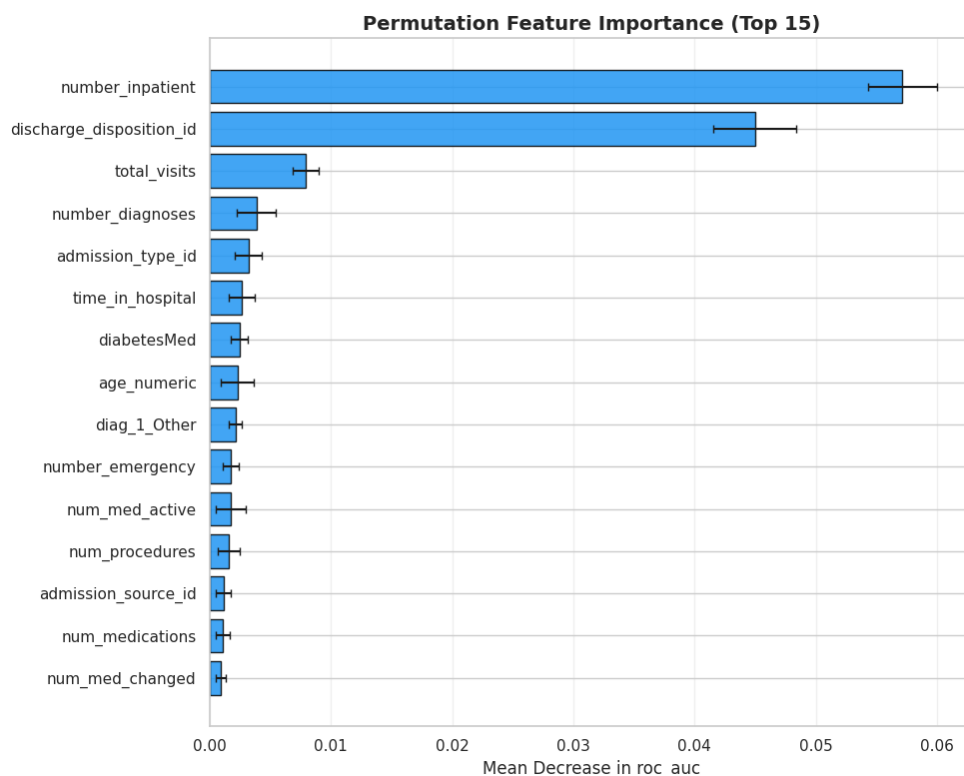


Fig 10. Permutation Feature Importance for the top 15 features. number_inpatient and discharge_disposition_id dominate.

Partial Dependence Plots confirmed the PFI results. Fig 11 shows that readmission probability increases sharply as the number of prior inpatient visits goes from 0 to about 3, and then continues to rise more gradually. This is clinically intuitive: patients with a history of frequent hospitalizations are more likely to be readmitted.

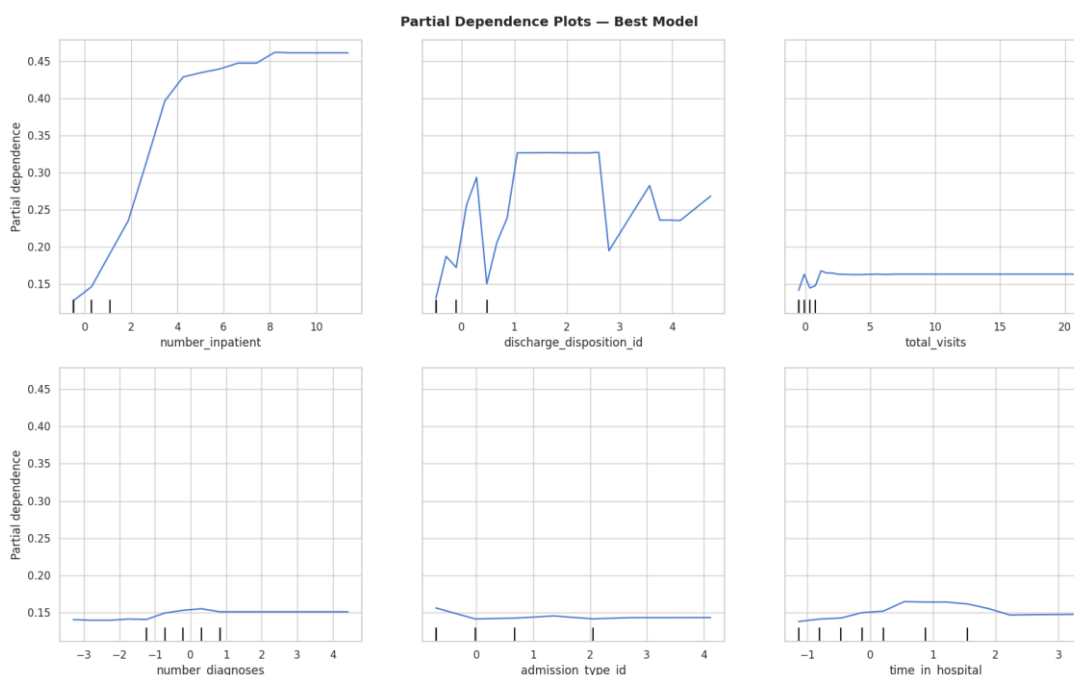


Fig 11. Partial Dependence Plots for the top features identified by PFI.

5.5 LACE Index vs. XGBoost Comparison

I computed LACE scores for all 19,868 patients in the test set and compared the results against the XGBoost model at the optimized threshold of 0.20. The Comorbidity component (C) of the LACE score was approximated using CCI-equivalent weights derived from the one-hot encoded diagnosis category columns: Diabetes=1, Neoplasms=2, Respiratory=1, Genitourinary=1. Table 6 shows the results.

Table 6. LACE Index vs. XGBoost on the same test set (n = 19,868).

Metric	LACE Index	XGBoost (T=0.20)	Improvement
ROC-AUC	0.5585	0.6648	+19.0%
Accuracy	87.95%	79.08%	-10.1%
Precision	20.89%	22.83%	+9.3%
Recall	2.08%	35.17%	+1,594%
F1-Score	0.0378	0.2769	+633%

XGBoost outperformed LACE on every metric that matters for catching readmissions: ROC-AUC, Precision, Recall, and F1-Score. LACE does show higher raw accuracy (87.95% vs. 79.08%), but this is the accuracy paradox at work. With a score distribution ranging from 1 to 12 (mean = 5.0), only 225 patients (1.1%) scored 10 or above, which is the standard LACE threshold for high risk. So LACE gets most predictions "right" by barely flagging anyone.

That said, LACE has a genuine advantage in simplicity: it requires no model training, works with just four inputs, and can be computed by hand at the bedside. XGBoost needs 46 features, a trained model, and a computer. The two approaches serve different needs, and the comparison shows where the ML model does better.

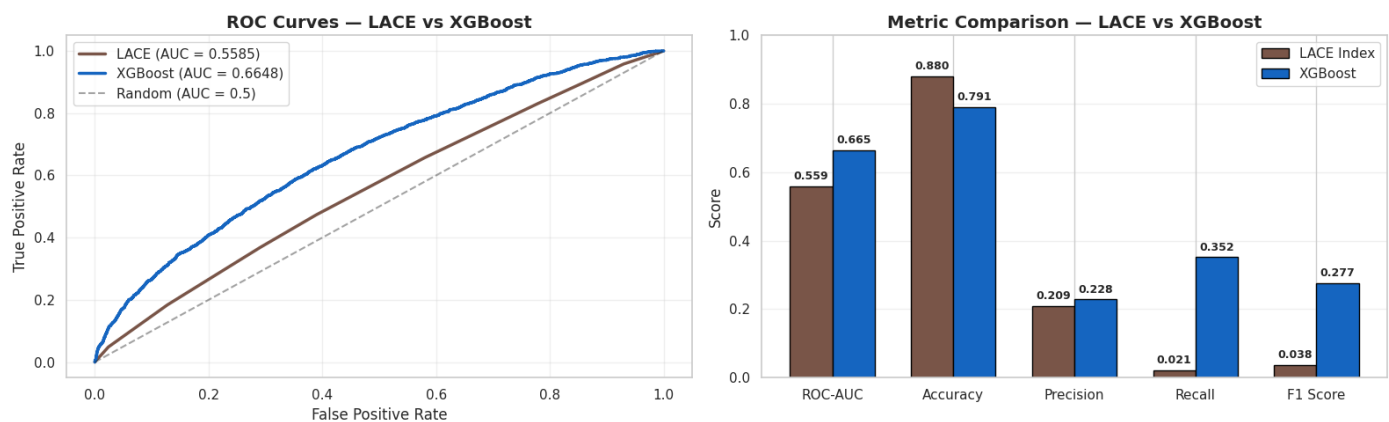


Fig 12. LACE vs. XGBoost: ROC curves (left) and metric comparison bar chart (right).

5.6 Streamlit Dashboard

The Streamlit dashboard (streamlit_app.py) lets clinical staff use the trained model without writing any code. It has four tabs:

- **Risk Assessment:** Users set patient parameters (age, medications, prior visits, diagnosis, etc.) using sliders and dropdowns, and it shows the predicted readmission probability, risk level (Low/Medium/High), and a visual risk gauge.
- **Explainability and What-If:** Shows which features are driving the prediction with z-score comparisons to the population average, and lets you simulate "what-if" scenarios by adjusting individual features.
- **Clinical Impact:** Covers the threshold optimization impact, a side-by-side LACE vs. XGBoost comparison for the current patient, and an estimated cost savings calculator.
- **Model Performance:** Shows the model comparison table along with a summary of the dataset and methodology.

The dashboard is deployed on Streamlit Community Cloud and auto-deploys from the GitHub repository on every push, so the live version always matches the latest code. The dashboard is accessible at: <https://cs719rajpanchal200490453.streamlit.app/>

5.7 Connection to Recent Research

The closest recent study is Guo et al. [2025], titled "Towards an Explainable Machine Learning Model to Reduce Readmission Risks for Diabetes Patients," published in Informatics in Medicine Unlocked. Their study used the same Diabetes 130-US Hospitals dataset and also focused on combining ML prediction with explainability techniques (PFI, PDP, and SHAP).

Their best model was a Bagging Random Forest, which achieved 89% accuracy and 67% precision. They tested nine classifiers (KNN, LR, Naive Bayes, LDA, QDA, AdaBoost, XGBoost, Decision Tree, and RF-Bagging) and compared three sampling strategies (none, random oversampling, and SMOTE). Their permutation importance results identified `number_inpatient` and `number_emergency` as the two most critical features, and their PDP plots showed a steady increase between `number_inpatient` and readmission probability.

Since both studies use the same dataset and the same target variable, some overlap is expected. Both identified `number_inpatient` as the top predictor, both used SMOTE for class imbalance, and both applied PFI and PDP for explainability. This overlap makes sense given the shared dataset and confirms that the results hold up.

That said, the two projects differ in several important ways:

- **Evaluation metric and model choice:** Guo et al. selected Bagging Random Forest based on accuracy (89%) and precision (67%). I selected XGBoost based on ROC-AUC (0.6648), which is a more reliable metric when one class heavily outnumbers the other, since accuracy can be misleading in that situation.
- **Number of classifiers and sampling strategies:** Guo et al. tested nine classifiers across three sampling strategies (no sampling, random oversampling, and SMOTE), giving a broader survey of techniques. I tested three classifiers (Logistic Regression, Random Forest, XGBoost) with SMOTE only, but spent more time on what happens after model selection.
- **Explainability techniques:** Guo et al. used PFI, PDP, and SHAP. I used PFI and PDP but did not include SHAP. However, my dashboard includes a patient-level what-if analysis that does something similar — it shows how changing one feature shifts the risk for a given patient.
- **Threshold optimization:** I swept thresholds from 0.05 to 0.95 and selected 0.20 to maximize F1-score, which improved recall from 4% to 35%. Guo et al. reported accuracy and precision but did not discuss threshold tuning, suggesting they used the default 0.50 cutoff.

- LACE Index comparison: I ran a head-to-head comparison between XGBoost and the LACE Index on the same held-out test set (19,868 patients). This comparison is absent from Guo et al. and provides a practical reference point, since LACE is what many hospitals actually use today.
- Deployment: I built and deployed a Streamlit web dashboard with real-time risk scoring, what-if analysis, and a live LACE comparison. Guo et al. did not include any deployment or user-facing component.
- Second-ranked feature: My PFI ranked `discharge_disposition_id` as the second most important feature, while Guo et al. found `number_emergency` in that position. Both make clinical sense, and the difference is probably due to the different models and preprocessing steps.

In short, both projects confirm that prior inpatient visits and discharge disposition are the strongest predictors of 30-day readmission, and that PFI and PDP work well for understanding what the model is doing. My project goes further with threshold tuning, the LACE benchmark, and a deployed dashboard.

6. Conclusions

This project set out to predict 30-day hospital readmission for diabetic patients and make those predictions explainable. Looking back at the goals described in Section 1, here is how it went:

I built a full machine learning pipeline using the Diabetes 130-US Hospitals dataset (101,766 encounters). After data cleaning, feature engineering, one-hot encoding, and SMOTE balancing, I trained and tuned three classifiers using GridSearchCV. XGBoost was selected as the best model with a test ROC-AUC of 0.6648. Threshold optimization moved the classification boundary from 0.50 to 0.20, increasing recall from 4% to 35%, which means the model now catches about nine times more actual readmissions than it did at the default threshold.

The explainability analysis using PFI and PDP identified prior inpatient visits (`number_inpatient`) and discharge disposition (`discharge_disposition_id`) as the two most important features. This makes sense clinically: patients who have been hospitalized before are more likely to come back, and where a patient goes after discharge (home, skilled nursing facility, etc.) strongly affects their readmission risk. This lines up with Guo et al. [2025], who independently identified the same top features using the same dataset.

The LACE Index comparison showed that XGBoost outperformed the rule-based clinical tool on ROC-AUC (0.6648 vs. 0.5585), Recall (35.2% vs. 2.1%), and F1-Score (0.2769 vs. 0.0378). LACE showed higher raw accuracy (87.95% vs. 79.08%), but this is the accuracy paradox: with only 1.1% of patients flagged as high risk, LACE gets most predictions right by barely flagging anyone. This shows that an ML model with more features can separate the classes better, but LACE is still useful because it is simple and easy to use.

Finally, the interactive Streamlit dashboard packages all of this into a tool that a clinician could actually use: real-time risk scores, what-if scenario analysis, feature explanations compared to the population average, and a side-by-side LACE breakdown for every patient.

The ROC-AUC of 0.6648, while moderate, is in line with the broader literature. Huang et al. [2021] reviewed 43 U.S.-based readmission studies and reported a median AUC of 0.68. Readmission depends on many factors not captured in structured hospital data, such as social support, patient compliance, and living conditions, so there is a ceiling on how well any model trained on EHR data alone can perform.

7. Suggestions for Future Research

The project met its goals, but there are a few areas that could be improved or extended:

- Constraints of the dataset and problem:
 - The dataset covers 1999 to 2008. Clinical practices, medications, and coding standards have changed a lot since then. Validating the model on more recent data would be important before any real clinical deployment.
 - The dataset only includes diabetic patients from U.S. hospitals. Results may not generalize to other patient populations, other countries, or non-diabetic readmissions.
 - The Charlson Comorbidity Index for the LACE comparison was approximated from the encoded diagnosis columns, not computed from the full ICD-9 code list. A dataset with actual CCI values would give a more accurate LACE comparison.
- Possible extensions:
 - EHR integration: The dashboard currently requires manual input of patient parameters. Connecting it to a hospital's Electronic Health Record system via an API would make it usable in a real clinical workflow without manual data entry.
 - Temporal modeling: The current approach treats each encounter independently. Using sequential models that account for a patient's history across multiple visits could capture patterns that single-encounter models miss.
 - External validation: Training on one hospital system and testing on another would give a more realistic estimate of how well the model generalizes.

8. References

All references listed below were consulted during the project. Each source was read and verified to support the claims made in this report.

- [1] Strack, B., DeShazo, J.P., Gennings, C., Olmo, J.L., Ventura, S., Cios, K.J., and Clore, J.N. (2014). Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records. BioMed Research International, 2014, Article 781670. Dataset available at: <https://archive.ics.uci.edu/ml/datasets/diabetes+130-us+hospitals+for+years+1999-2008>
- [2] van Walraven, C., Dhalla, I.A., Bell, C., Etchells, E., Stiell, I.G., Zarnke, K., Austin, P.C., and Forster, A.J. (2010). Derivation and Validation of an Index to Predict Early Death or Unplanned Readmission After Discharge from Hospital to the Community. Canadian Medical Association Journal (CMAJ), 182(6), 551-557. <https://doi.org/10.1503/cmaj.091117>
- [3] Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794. <https://doi.org/10.1145/2939672.2939785>
- [4] Chawla, N.V., Bowyer, K.W., Hall, L.O., and Kegelmeyer, W.P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357. <https://doi.org/10.1613/jair.953>
- [5] Guo, C., Zhou, H., Pires, I.M., Coelho, P.J., Tong, R., and Farid, F. (2025). Towards an Explainable Machine Learning Model to Reduce Readmission Risks for Diabetes Patients. Informatics in Medicine Unlocked, 58, 101686. <https://doi.org/10.1016/j.imu.2025.101686>
- [6] Huang, Y., Talwar, A., Chatterjee, S., and Aparasu, R.R. (2021). Application of Machine Learning in Predicting Hospital Readmissions: A Scoping Review of the Literature. BMC Medical Research Methodology, 21, 96. <https://doi.org/10.1186/s12874-021-01284-z>
- [7] Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.). Available at: <https://christophm.github.io/interpretable-ml-book/>
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. Available at: <https://scikit-learn.org/stable/>
- [9] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>